SIMATS ENGINEERING

ASSESSMENT TOOL 1

DESCRIPTIVE EXPERIMENT EXAM

Course Code	MLA03
Course Name	Reinforcement learning for Environment and Agent
Course Outcome	CO1: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems (BL2, BL3)
Weightage / Marks	40% | 10 Experiments | 50 Marks | Duration: 2 hrs
Faculty	K. VEENA DEVI
Student Name / Reg. No.	Aryan.K.S 192325051

Environment used for all experiments: Python 3.12, NumPy 2.4, Matplotlib 3.10, Gymnasium 1.4, TensorFlow 2.21, Keras 3.15. All programs were executed and the outputs shown are the actual results obtained.

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim

To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib and other required libraries, and to verify the environment by executing a simple Reinforcement Learning program.

Procedure / Algorithm

1. Download and install Anaconda (Python 3.x) from anaconda.com and open Anaconda Prompt / Navigator.
2. Create an isolated environment: conda create -n rl_env python=3.12 -y, then conda activate rl_env.
3. Install the libraries: pip install numpy matplotlib gymnasium tensorflow keras jupyter.
4. Launch Jupyter Notebook (or Spyder) from Anaconda Navigator inside rl_env.
5. Run the verification program below: print library versions and let a random agent interact with CartPole-v1.

Program

```
import sys, platform
import numpy as np, matplotlib, gymnasium as gym
import tensorflow as tf, keras
print("Python      :", platform.python_version())
print("NumPy       :", np.__version__)
print("Matplotlib  :", matplotlib.__version__)
print("Gymnasium   :", gym.__version__)
print("TensorFlow  :", tf.__version__)
print("Keras       :", keras.__version__)
env = gym.make("CartPole-v1")
obs, info = env.reset(seed=42)
total = 0
```

```
for t in range(200):
    obs, r, term, trunc, info = env.step(env.action_space.sample())
    total += r
    if term or trunc: break
print("Test RL program: random agent on CartPole-v1")
print("Steps survived:", t+1, "| Total reward:", total)
print("Environment verified successfully.")
```

Output

```
Python      : 3.12.3
NumPy       : 2.4.4
Matplotlib  : 3.10.8
Gymnasium   : 1.4.0
TensorFlow  : 2.21.0
Keras       : 3.15.1
Test RL program: random agent on CartPole-v1
Steps survived: 20 | Total reward: 20.0
Environment verified successfully.
```

Result

The RL development environment was installed successfully. All libraries imported without errors and the random agent completed an episode in CartPole-v1, which verifies the setup.

Experiment 2: Familiarization with OpenAI Gym / Gymnasium Environments

Aim

To create, initialize and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole and MountainCar using Gymnasium, to understand the concepts of states, actions, rewards and episodes.

Procedure / Algorithm

1. Import gymnasium and create each environment with gym.make().
2. Call env.reset() to obtain the initial state (observation).
3. Inspect env.observation_space and env.action_space to see the state and action definitions.
4. Repeatedly call env.step(action) to receive next state, reward, terminated and truncated flags.
5. Stop when the episode ends and record the number of steps and total reward.

Program

```
import gymnasium as gym
for name in ["FrozenLake-v1","CartPole-v1","MountainCar-v0"]:
    env = gym.make(name) if name!="FrozenLake-v1" else gym.make(name, is_slippery=False)
    obs, _ = env.reset(seed=1)
    print("="*55); print("Environment :", name)
    print("Observation space:", env.observation_space)
    print("Action space     :", env.action_space)
    print("Initial state    :", obs)
    total, steps = 0, 0
    for _ in range(500):
        a = env.action_space.sample()
        obs, r, term, trunc, _ = env.step(a)
        total += r; steps += 1
        if steps <= 3: print(f"  step {steps}: action={a}, next_state={obs}, reward={r}")
        if term or trunc: break
    print(f"Episode ended after {steps} steps, total reward = {total}")
    env.close()
```

Output

```
======================================================
Environment : FrozenLake-v1
Observation space: Discrete(16)
Action space     : Discrete(4)
Initial state    : 0
  step 1: action=3, next_state=0, reward=0
  step 2: action=1, next_state=4, reward=0
  step 3: action=3, next_state=0, reward=0
Episode ended after 12 steps, total reward = 0
======================================================
Environment : CartPole-v1
Observation space: Box([-4.8               -inf -0.41887903        -inf], [4.8               inf
0.41887903...
Action space     : Discrete(2)
Initial state    : [ 0.00118216  0.04504637 -0.03558404  0.04486495]
  step 1: action=0, next_state=[ 0.00208309 -0.14954773 -0.03468674  0.32611182], reward=1.0
  step 2: action=1, next_state=[-0.00090786  0.04605047 -0.0281645   0.02269515], reward=1.0
  step 3: action=1, next_state=[ 1.3144526e-05  2.4156477e-01 -2.7710600e-02 -2.7873927e-01],
reward=1.0
Episode ended after 18 steps, total reward = 18.0
======================================================
Environment : MountainCar-v0
Observation space: Box([-1.2  -0.07], [0.6  0.07], (2,), float32)
Action space     : Discrete(3)
Initial state    : [-0.49763566  0.        ]
  step 1: action=0, next_state=[-0.4988302  -0.00119453], reward=-1.0
  step 2: action=0, next_state=[-0.50121033 -0.00238012], reward=-1.0
  step 3: action=2, next_state=[-0.5027582  -0.00154791], reward=-1.0
Episode ended after 200 steps, total reward = -200.0
```

Result

FrozenLake has a discrete state space (16 states) with 4 actions and a sparse reward; CartPole has a continuous 4-dimensional state with 2 actions and +1 reward per step; MountainCar has a continuous 2-dimensional state with 3 actions and -1 reward per step. The terms state, action, reward and episode were understood by interacting with all three environments.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim

To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards and policies using Python.

Procedure / Algorithm

1. Define an Environment class with reset() and step(action) returning (next_state, reward, done).
2. Define an Agent class holding a Q-table, an epsilon-greedy policy and a learning (Q-update) rule.
3. Run the agent-environment interaction loop for several episodes: observe state, choose action, receive reward, update Q-values.
4. Extract the greedy policy from the learned Q-table and display it.

Program

```
import numpy as np, random
class GridEnv:
    """4x4 grid; start (0,0); goal (3,3); pit (1,1)... actions 0=U 1=R 2=D 3=L"""
    def __init__(self): self.n=4; self.reset()
    def reset(self): self.pos=(0,0); return self.pos
    def step(self,a):
        r,c=self.pos
```

```
        dr,dc=[(-1,0),(0,1),(1,0),(0,-1)][a]
        r=min(max(r+dr,0),self.n-1); c=min(max(c+dc,0),self.n-1)
        self.pos=(r,c)
        if self.pos==(3,3): return self.pos,10,True
        if self.pos==(1,1): return self.pos,-10,True
        return self.pos,-1,False
class Agent:
    def __init__(self,alpha=0.1,gamma=0.95,eps=0.2):
        self.Q=np.zeros((4,4,4)); self.alpha,self.gamma,self.eps=alpha,gamma,eps
    def policy(self,s):                      # epsilon-greedy policy
        if random.random()<self.eps: return random.randrange(4)
        return int(np.argmax(self.Q[s]))
    def learn(self,s,a,r,s2,done):           # Q-learning update
        tgt=r+(0 if done else self.gamma*np.max(self.Q[s2]))
        self.Q[s][a]+=self.alpha*(tgt-self.Q[s][a])
random.seed(0); np.random.seed(0)
env,agent=GridEnv(),Agent()
for ep in range(1,501):
    s,done,G=env.reset(),False,0
    while not done:
        a=agent.policy(s); s2,r,done=env.step(a); agent.learn(s,a,r,s2,done); s=s2; G+=r
    if ep%100==0: print(f"Episode {ep:3d} | return = {G}")
print("\nLearned greedy policy (U/R/D/L):")
sym="URDL"
for r in range(4):
    print(" ".join("G" if (r,c)==(3,3) else "X" if (r,c)==(1,1) else
sym[int(np.argmax(agent.Q[r,c]))] for c in range(4)))
```

Output

```
Episode 100 | return = 5
Episode 200 | return = 5
Episode 300 | return = 3
Episode 400 | return = 1
Episode 500 | return = 4

Learned greedy policy (U/R/D/L):
R R D D
D X D D
R R R D
R R R G
```

Result

The agent-environment loop was implemented and the agent learned a policy that moves from the start cell to the goal G while avoiding the pit X, demonstrating the state-action-reward-policy cycle.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim

To develop a simple Markov Decision Process model and simulate state transitions, reward functions and policy execution for understanding sequential decision-making problems.

Procedure / Algorithm

1. Define states (Low, Medium, High) and actions (Wait, Work).
2. Define the transition probability matrix P(s'|s,a) and the reward function R(s,a).
3. Choose a fixed policy mapping each state to an action.
4. Simulate a trajectory by sampling next states from P and accumulate the discounted return with gamma = 0.9.
5. Repeat for 2000 episodes to estimate the expected discounted return.

Program

```
import numpy as np
```

```
np.random.seed(7)
states=["Low","Medium","High"]; actions=["Wait","Work"]
# P[a][s][s']  and R[a][s]
P={0:np.array([[0.8,0.2,0.0],[0.3,0.6,0.1],[0.0,0.4,0.6]]),
   1:np.array([[0.3,0.6,0.1],[0.1,0.4,0.5],[0.0,0.2,0.8]])}
R={0:np.array([0,1,2]),1:np.array([-1,0,3])}
policy={0:1,1:1,2:1}              # fixed policy: always Work... modify below
policy={0:1,1:0,2:1}
print("MDP: states =",states,"| actions =",actions)
print("Policy:",{states[s]:actions[a] for s,a in policy.items()})
s=0; G=0; gamma=0.9
print("\nSimulated trajectory (12 steps):")
for t in range(12):
    a=policy[s]; s2=np.random.choice(3,p=P[a][s]); r=R[a][s]
    G+=(gamma**t)*r
    print(f"t={t:2d} | {states[s]:6s} --{actions[a]:4s}--> {states[s2]:6s} | reward={r:2d}")
    s=s2
print(f"\nDiscounted return (gamma={gamma}): {G:.3f}")
# average over many runs
rets=[]
for _ in range(2000):
    s=0;g=0
    for t in range(30):
        a=policy[s]; s2=np.random.choice(3,p=P[a][s]); g+=(gamma**t)*R[a][s]; s=s2
    rets.append(g)
print(f"Mean discounted return over 2000 episodes: {np.mean(rets):.3f}")
```

Output

```
MDP: states = ['Low', 'Medium', 'High'] | actions = ['Wait', 'Work']
Policy: {'Low': 'Work', 'Medium': 'Wait', 'High': 'Work'}

Simulated trajectory (12 steps):
t= 0 | Low    --Work--> Low    | reward=-1
t= 1 | Low    --Work--> Medium | reward=-1
t= 2 | Medium --Wait--> Medium | reward= 1
t= 3 | Medium --Wait--> Medium | reward= 1
t= 4 | Medium --Wait--> High   | reward= 1
t= 5 | High   --Work--> High   | reward= 3
t= 6 | High   --Work--> High   | reward= 3
t= 7 | High   --Work--> Medium | reward= 3
t= 8 | Medium --Wait--> Low    | reward= 1
t= 9 | Low    --Work--> Medium | reward=-1
t=10 | Medium --Wait--> Medium | reward= 1
t=11 | Medium --Wait--> Medium | reward= 1

Discounted return (gamma=0.9): 5.801
Mean discounted return over 2000 episodes: 8.242
```

Result

The MDP was simulated successfully. State transitions followed the defined probabilities and the average discounted return of the policy was estimated as about 8.24.

Experiment 5: Bellman Equation Demonstration

Aim

To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and to analyze the convergence of value estimation.

Procedure / Algorithm

1. Create a 4x4 grid-world with terminal states at (0,0) and (3,3) and reward -1 per step.
2. Bellman Expectation: V(s) = sum_a pi(a|s) [r + gamma V(s')] with an equiprobable random policy, iterated until the maximum change is below 1e-6.
3. Bellman Optimality: V*(s) = max_a [r + gamma V*(s')] (value iteration), iterated until convergence.

4. Compute the optimal action values Q*(s,a) and plot the convergence of both methods.

Program

```
import numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
# 4x4 gridworld, terminal at (0,0) and (3,3), reward -1 per step, gamma=1
n=4; gamma=1.0; moves=[(-1,0),(0,1),(1,0),(0,-1)]
term={(0,0),(3,3)}
def step(s,a):
    if s in term: return s,0
    r,c=s; dr,dc=moves[a]; r2,c2=r+dr,c+dc
    if not(0<=r2<n and 0<=c2<n): r2,c2=r,c
    return (r2,c2),-1
# Bellman expectation: equiprobable random policy
V=np.zeros((n,n)); deltas=[]
for k in range(1,1000):
    Vn=V.copy(); d=0
    for r in range(n):
        for c in range(n):
            s=(r,c)
            if s in term: continue
            v=sum(0.25*(rew+gamma*V[s2]) for a in range(4) for s2,rew in [step(s,a)])
            Vn[r,c]=v; d=max(d,abs(v-V[r,c]))
    V=Vn; deltas.append(d)
    if d<1e-6: break
print("Bellman EXPECTATION (random policy) converged in",k,"sweeps")
print(np.round(V,2))
# Bellman optimality: value iteration
Vo=np.zeros((n,n)); do=[]
for k2 in range(1,1000):
    Vn=Vo.copy(); d=0
    for r in range(n):
        for c in range(n):
            s=(r,c)
            if s in term: continue
            v=max(rew+gamma*Vo[s2] for a in range(4) for s2,rew in [step(s,a)])
            Vn[r,c]=v; d=max(d,abs(v-Vo[r,c]))
    Vo=Vn; do.append(d)
    if d<1e-6: break
print("\nBellman OPTIMALITY (value iteration) converged in",k2,"sweeps")
print(np.round(Vo,2))
# optimal action values Q*(s,a)
print("\nOptimal Q*(s,a) for state (0,1) [U,R,D,L]:",[round(float(rew+gamma*Vo[s2]),1) for a in
range(4) for s2,rew in [step((0,1),a)]])
fig,ax=plt.subplots(1,2,figsize=(9,3.4))
ax[0].plot(deltas,label="Bellman expectation"); ax[0].plot(do,label="Bellman optimality")
ax[0].set_yscale("log"); ax[0].set_xlabel("Sweep"); ax[0].set_ylabel("Max value change");
ax[0].set_title("Convergence of value estimation"); ax[0].legend()
im=ax[1].imshow(Vo,cmap="viridis"); ax[1].set_title("Optimal state values V*")
for r in range(n):
    for c in range(n): ax[1].text(c,r,f"{Vo[r,c]:.0f}",ha="center",va="center",color="w")
plt.tight_layout(); plt.savefig("figs/exp5.png",dpi=150)
```

Output

```
Bellman EXPECTATION (random policy) converged in 258 sweeps
[[  0. -14. -20. -22.]
 [-14. -18. -20. -20.]
 [-20. -20. -18. -14.]
 [-22. -20. -14.   0.]]

Bellman OPTIMALITY (value iteration) converged in 4 sweeps
[[ 0. -1. -2. -3.]
 [-1. -2. -3. -2.]
 [-2. -3. -2. -1.]
 [-3. -2. -1.  0.]]

Optimal Q*(s,a) for state (0,1) [U,R,D,L]: [-2.0, -3.0, -3.0, -1.0]
```

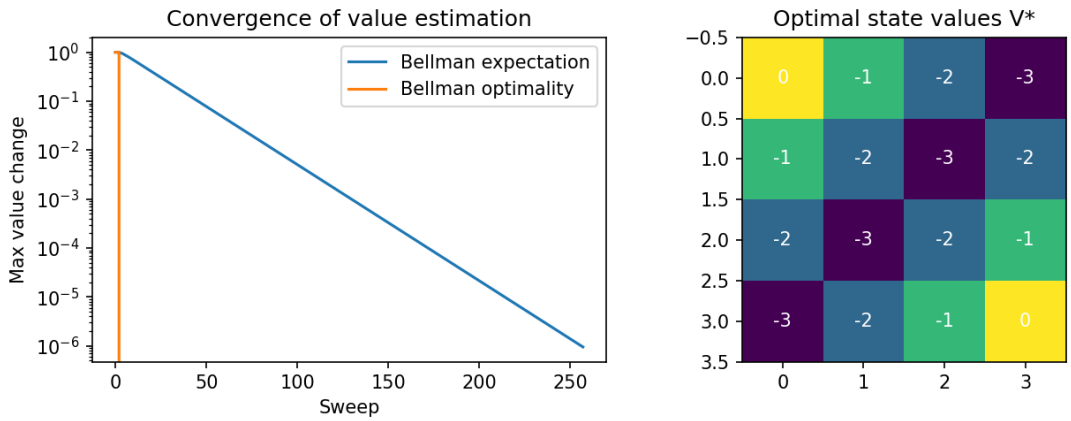


Figure 5: Output graph of Experiment 5

Result

Policy evaluation of the random policy needed 258 sweeps to converge whereas value iteration for the optimal Bellman equation converged in only 4 sweeps. The optimal value of each cell equals the negative of its shortest-path distance to a terminal state, as expected.

Experiment 6: Exploration vs. Exploitation Strategies

Aim

To implement Epsilon-Greedy, Greedy and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Procedure / Algorithm

1. Create a 10-armed bandit testbed with true action values drawn from N(0,1) and noisy rewards.
2. Implement three action-selection strategies: Random, Greedy and epsilon-Greedy (eps = 0.01 and 0.1).
3. Use the sample-average update Q(a) = Q(a) + (r - Q(a))/N(a).
4. Run 1000 steps and average over 500 independent runs; record average reward and % optimal action.

Program

```
import numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
K=10; T=1000; RUNS=500
rng=np.random.default_rng(0)
def run(strategy,eps=0.1):
    q_true=rng.normal(0,1,K); Q=np.zeros(K); N=np.zeros(K); rewards=np.zeros(T); opt=np.zeros(T)
    best=np.argmax(q_true)
    for t in range(T):
        if strategy=="random": a=rng.integers(K)
        elif strategy=="greedy": a=int(np.argmax(Q))
        else: a=rng.integers(K) if rng.random()<eps else int(np.argmax(Q))
        r=rng.normal(q_true[a],1); N[a]+=1; Q[a]+=(r-Q[a])/N[a]
        rewards[t]=r; opt[t]=(a==best)
    return rewards,opt
res={}
for name,kw in [("Random",("random",0)),("Greedy",("greedy",0)),("eps-greedy
(0.01)",("eps",0.01)),("eps-greedy (0.1)",("eps",0.1))]:
    R=np.zeros(T);O=np.zeros(T)
    for _ in range(RUNS):
        r,o=run(*kw); R+=r;O+=o
    res[name]=(R/RUNS,O/RUNS)
print(f"{'Strategy':20s} {'Avg reward':>11s} {'% optimal action':>17s}")
for n,(R,O) in res.items(): print(f"{n:20s} {R.mean():11.3f} {O[-200:].mean()*100:16.1f}%")
fig,ax=plt.subplots(1,2,figsize=(10,3.6))
```

```
for n,(R,O) in res.items():
    ax[0].plot(np.cumsum(R)/np.arange(1,T+1),label=n); ax[1].plot(O*100,label=n)
ax[0].set_title("Average reward"); ax[0].set_xlabel("Steps"); ax[0].legend(fontsize=7)
ax[1].set_title("% optimal action"); ax[1].set_xlabel("Steps")
plt.tight_layout(); plt.savefig("figs/exp6.png",dpi=150)
```

Output

```
Strategy             Avg reward  % optimal action
Random                    0.017             10.1%
Greedy                    1.011             37.0%
eps-greedy (0.01)         1.206             57.4%
eps-greedy (0.1)          1.257             76.7%
```

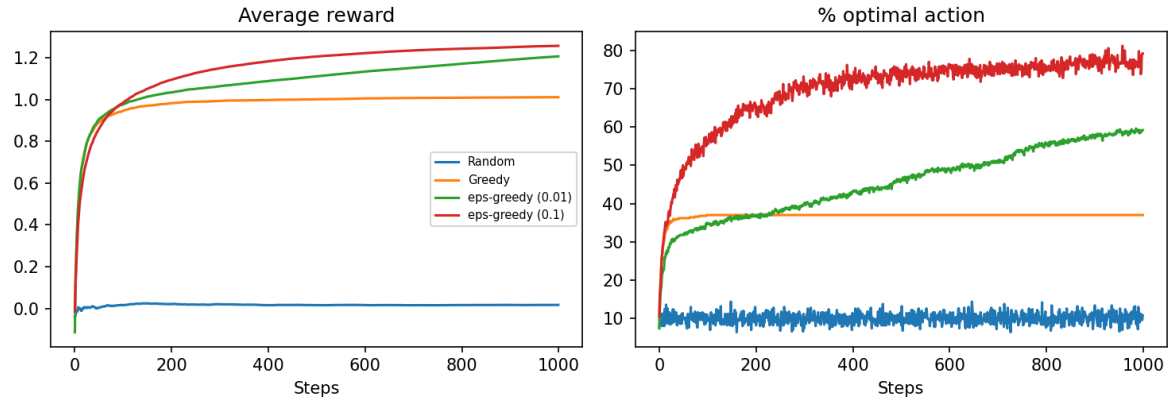


Figure 6: Output graph of Experiment 6

Result

Random selection never exploits (average reward about 0.02). Pure Greedy gets stuck on sub-optimal arms (37% optimal actions). Epsilon-Greedy with eps = 0.1 gave the best balance, with the highest average reward (1.26) and 76.7% optimal actions.

Experiment 7: Multi-Armed Bandit Problem

Aim

To implement the Multi-Armed Bandit problem using Epsilon-Greedy and Upper Confidence Bound (UCB) algorithms and to evaluate the cumulative rewards obtained by different action-selection methods.

Procedure / Algorithm

1. Create a 10-armed Bernoulli bandit with success probabilities drawn uniformly from [0.1, 0.9].
2. Epsilon-Greedy: explore randomly with probability 0.1, otherwise choose argmax Q(a).
3. UCB: choose argmax [Q(a) + c * sqrt(ln t / N(a))] for c = 0.5, 1.0 and 2.0.
4. Run 2000 steps, average over 300 runs, and compare cumulative reward and cumulative regret.

Program

```
import numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
K=10; T=2000; RUNS=300; rng=np.random.default_rng(1)
def run(method,c=1.0):
    p=rng.uniform(0.1,0.9,K); Q=np.zeros(K); N=np.zeros(K); cum=np.zeros(T); reg=np.zeros(T);
tot=0; rg=0
    for t in range(T):
        if method=="eps": a=rng.integers(K) if rng.random()<0.1 else int(np.argmax(Q))
        else: a=t if t<K else int(np.argmax(Q+c*np.sqrt(np.log(t+1)/N)))
```

```
        r=float(rng.random()<p[a]); N[a]+=1; Q[a]+=(r-Q[a])/N[a]; tot+=r; rg+=p.max()-p[a]
        cum[t]=tot; reg[t]=rg
    return cum,reg
cfg=[("eps","epsilon-Greedy (eps=0.1)",0),("ucb","UCB (c=0.5)",0.5),("ucb","UCB
(c=1.0)",1.0),("ucb","UCB (c=2.0)",2.0)]
out={}
for m,label,c in cfg:
    C=np.zeros(T);G=np.zeros(T)
    for _ in range(RUNS):
        cc,g=run(m,c); C+=cc;G+=g
    out[label]=(C/RUNS,G/RUNS)
print(f"{'Method':26s} {'Cum. reward':>12s} {'Cum. regret':>12s}   (T={T}, {RUNS} runs, 10
Bernoulli arms)")
for l,(C,G) in out.items(): print(f"{l:26s} {C[-1]:12.1f} {G[-1]:12.1f}")
fig,ax=plt.subplots(1,2,figsize=(10,3.6))
for l,(C,G) in out.items(): ax[0].plot(C,label=l); ax[1].plot(G,label=l)
ax[0].set_title("Cumulative reward"); ax[1].set_title("Cumulative regret")
for a in ax: a.set_xlabel("Steps"); a.legend(fontsize=8)
plt.tight_layout(); plt.savefig("figs/exp7.png",dpi=150)
```

Output

```
Method                       Cum. reward  Cum. regret   (T=2000, 300 runs, 10 Bernoulli arms)
epsilon-Greedy (eps=0.1)          1548.2        110.8
UCB (c=0.5)                       1603.7         51.1
UCB (c=1.0)                       1514.9        127.3
UCB (c=2.0)                       1386.3        266.9
```

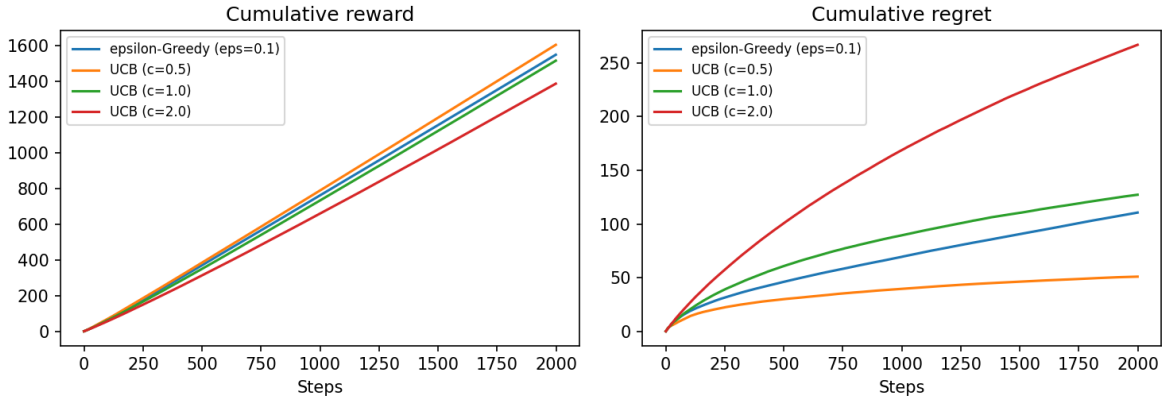


Figure 7: Output graph of Experiment 7

Result

UCB with a well-chosen exploration constant (c = 0.5) obtained the highest cumulative reward (1603.7) and the lowest regret (51.1), better than epsilon-Greedy (1548.2). A very large constant (c = 2) over-explores and performs worse, showing that the exploration parameter must be tuned.

Experiment 8: Reward Visualization in Reinforcement Learning

Aim

To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards and learning performance using Matplotlib graphs.

Procedure / Algorithm

1. Train a tabular Q-learning agent on FrozenLake-v1 (non-slippery) for 1000 episodes with decaying epsilon.
2. Store the reward of every episode.
3. Plot (a) episode reward, (b) cumulative reward and (c) the 50-episode moving average using Matplotlib.

Program

```
import numpy as np, gymnasium as gym, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot
as plt
env=gym.make("FrozenLake-v1",is_slippery=False); rng=np.random.default_rng(3)
Q=np.zeros((16,4)); alpha,gamma=0.8,0.95; E=1000; rewards=[]
for ep in range(E):
    eps=max(0.05,1-ep/500); s,_=env.reset(); done=False; G=0
    while not done:
        a=rng.integers(4) if rng.random()<eps else int(np.argmax(Q[s]+rng.random(4)*1e-6))
        s2,r,term,trunc,_=env.step(a); done=term or trunc
        Q[s,a]+=alpha*(r+gamma*np.max(Q[s2])*(not term)-Q[s,a]); s=s2; G+=r
    rewards.append(G)
rewards=np.array(rewards); cum=np.cumsum(rewards);
ma=np.convolve(rewards,np.ones(50)/50,mode="valid")
print("Total episodes          :",E)
print("Total cumulative reward :",int(cum[-1]))
print("Success rate (first 100):",rewards[:100].mean())
print("Success rate (last 100) :",rewards[-100:].mean())
fig,ax=plt.subplots(1,3,figsize=(12,3.4))
ax[0].plot(rewards,lw=.4); ax[0].set_title("Episode reward")
ax[1].plot(cum); ax[1].set_title("Cumulative reward")
ax[2].plot(ma); ax[2].set_title("Moving average (50 episodes)")
for a in ax: a.set_xlabel("Episode")
plt.tight_layout(); plt.savefig("figs/exp8.png",dpi=150)
```

Output

```
Total episodes          : 1000
Total cumulative reward : 632
Success rate (first 100): 0.0
Success rate (last 100) : 0.9
```

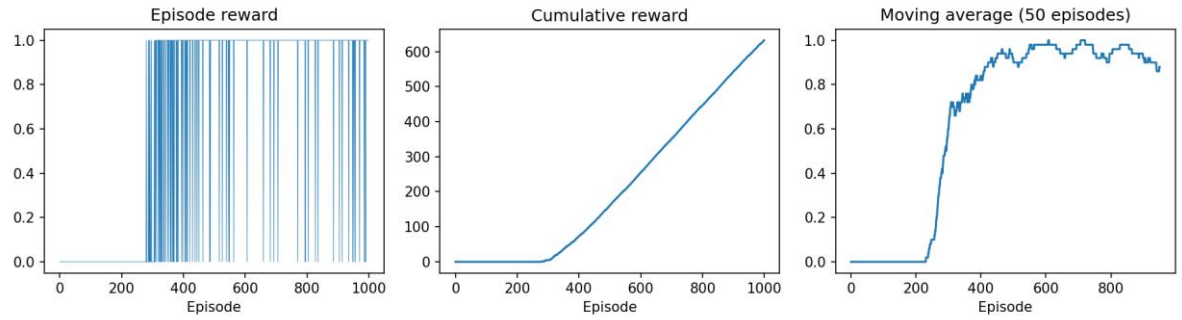


Figure 8: Output graph of Experiment 8

Result

The graphs show that the agent received no reward in the early exploration phase and then learned to reach the goal consistently: the success rate rose from 0% in the first 100 episodes to 90% in the last 100 episodes.

Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim

To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Procedure / Algorithm

1. Create a 6x6 grid-world and compute the true state values V*(s) using value iteration (gamma = 0.9).
2. Use the normalized (row, column) coordinates as the network input and V*(s) as the target.
3. Build a Keras Sequential network: Dense(32, ReLU) - Dense(32, ReLU) - Dense(1).
4. Train with the Adam optimizer and mean-squared-error loss, then compare the predicted values with the true values.

Program

```
import os; os.environ["TF_CPP_MIN_LOG_LEVEL"]="3"
import numpy as np, tensorflow as tf, keras, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
from keras import layers
np.random.seed(0); tf.random.set_seed(0); keras.utils.set_random_seed(0)
n=6; gamma=0.9; goal=(5,5); moves=[(-1,0),(0,1),(1,0),(0,-1)]
def step(s,a):
    r,c=s; dr,dc=moves[a]; r2,c2=min(max(r+dr,0),n-1),min(max(c+dc,0),n-1)
    s2=(r2,c2); return s2,(10 if s2==goal else -0.1)
V=np.zeros((n,n))
for _ in range(200):                      # value iteration gives target V*(s)
    Vn=V.copy()
    for r in range(n):
        for c in range(n):
            if (r,c)==goal: continue
            Vn[r,c]=max(rew+gamma*V[s2] for a in range(4) for s2,rew in [step((r,c),a)])
    V=Vn
X=np.array([[r/(n-1),c/(n-1)] for r in range(n) for c in range(n) if (r,c)!=goal],dtype="float32")
y=np.array([V[r,c] for r in range(n) for c in range(n) if (r,c)!=goal],dtype="float32")
model=keras.Sequential([layers.Input((2,)),layers.Dense(32,activation="relu"),layers.Dense(32,activation="relu"),
model.compile(optimizer=keras.optimizers.Adam(0.01),loss="mse")
model.summary(print_fn=lambda s:print(s))
h=model.fit(X,y,epochs=1500,batch_size=12,verbose=0)
Xall=np.array([[r/(n-1),c/(n-1)] for r in range(n) for c in range(n)],dtype="float32");
pred=model.predict(Xall,verbose=0).reshape(n,n); pred[goal]=0.0
print("\nFinal training MSE:",round(float(h.history['loss'][-1]),5))
print("MAE vs true V* (non-terminal states):",round(float(np.delete(np.abs(pred-V).ravel(),n*n-1).mean()),4))
print("\nTrue V* (value iteration):"); print(np.round(V,2))
print("\nNN-approximated V(s):");    print(np.round(pred,2))
fig,ax=plt.subplots(1,3,figsize=(12,3.4))
ax[0].plot(h.history["loss"]); ax[0].set_yscale("log"); ax[0].set_title("Training loss (MSE)"); ax[0].set_xlabel(
for a,M,t in [(ax[1],V,"True V*"),(ax[2],pred,"Neural-network V(s)")]:
    a.imshow(M,cmap="viridis"); a.set_title(t)
    for r in range(n):
        for c in range(n): a.text(c,r,f"{M[r,c]:.1f}",ha="center",va="center",color="w",fontsize=7)
plt.tight_layout(); plt.savefig("figs/exp9.png",dpi=150)
```

Output

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32)	96
dense_1 (Dense)	(None, 32)	1,056
dense_2 (Dense)	(None, 1)	33

```
Total params: 1,185 (4.63 KB)
Trainable params: 1,185 (4.63 KB)
```

```
 Non-trainable params: 0 (0.00 B)


Final training MSE: 0.00375
MAE vs true V* (non-terminal states): 0.0499

True V* (value iteration):
[[ 3.26  3.74  4.26  4.85  5.5   6.22]
 [ 3.74  4.26  4.85  5.5   6.22  7.02]
 [ 4.26  4.85  5.5   6.22  7.02  7.91]
 [ 4.85  5.5   6.22  7.02  7.91  8.9 ]
 [ 5.5   6.22  7.02  7.91  8.9  10.  ]
 [ 6.22  7.02  7.91  8.9  10.    0.  ]]

NN-approximated V(s):
[[3.18 3.75 4.32 4.88 5.48 6.26]
 [3.71 4.31 4.88 5.45 6.13 7.06]
 [4.28 4.88 5.48 6.14 7.04 7.96]
 [4.87 5.46 6.15 7.09 8.02 8.92]
 [5.51 6.12 7.07 8.02 8.96 9.9 ]
 [6.23 7.05 7.98 8.93 9.89 0.  ]]
```

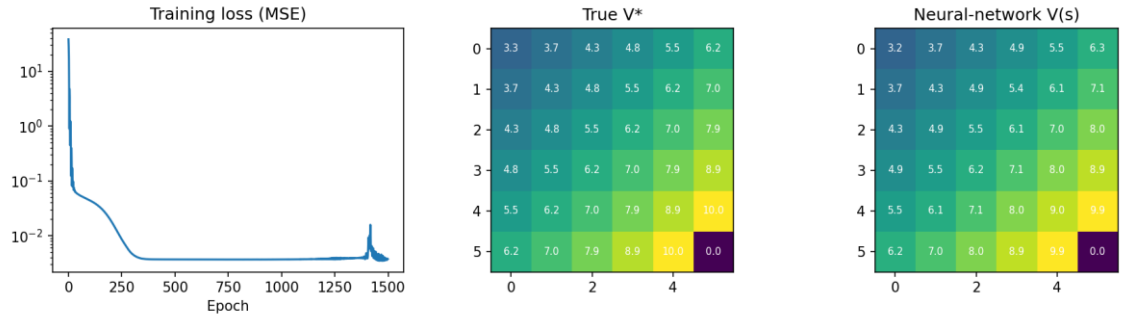


Figure 9: Output graph of Experiment 9

Result

The feed-forward network approximated the value function accurately (final MSE = 0.0038, mean absolute error = 0.05), proving that neural networks can generalise value estimates over states, which is the basis of Deep Q-Networks.

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim

To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and to evaluate its learning performance through episode rewards and success rate.

Procedure / Algorithm

1. Build a Q-network in Keras (4 inputs, two Dense(64, ReLU) layers, 2 outputs) and a separate target network.
2. Use an epsilon-greedy policy with decaying epsilon and an experience-replay buffer.
3. Train with the Deep Q-Network (DQN) target r + gamma * max Q_target(s',a') for 400 episodes.
4. Evaluate the learned greedy policy for 20 episodes and compute the success rate (reward >= 195).

Program

```
import os; os.environ["TF_CPP_MIN_LOG_LEVEL"]="3"
```

```
import random, time, numpy as np, gymnasium as gym, tensorflow as tf, keras, matplotlib; matplotlib.use("Agg"); i
matplotlib.pyplot as plt
from collections import deque
from keras import layers
random.seed(0); np.random.seed(0); keras.utils.set_random_seed(0)
env=gym.make("CartPole-v1")
def build():

m=keras.Sequential([layers.Input((4,)),layers.Dense(64,activation="relu"),layers.Dense(64,activation="relu"),laye
    m.compile(optimizer=keras.optimizers.Adam(1e-3),loss="mse"); return m
q,target=build(),build(); target.set_weights(q.get_weights())
mem=deque(maxlen=20000); gamma=0.99; eps=1.0; eps_min=0.02; eps_decay=0.985; bs=64
fast_q=tf.function(lambda x:q(x,training=False))
EPISODES=int(os.environ.get("EPISODES",400)); rewards=[]; t0=time.time(); steps=0
for ep in range(1,EPISODES+1):
    s,_=env.reset(); done=False; G=0
    while not done:
        if random.random()<eps: a=env.action_space.sample()
        else: a=int(np.argmax(fast_q(s.reshape(1,-1).astype("float32")).numpy()[0]))
        s2,r,term,trunc,_=env.step(a); done=term or trunc
        mem.append((s,a,r,s2,float(term))); s=s2; G+=r; steps+=1
        if len(mem)>=bs and steps%2==0:
            b=random.sample(mem,bs); S,A,R,S2,D=map(np.array,zip(*b))
            tq=q.predict_on_batch(S.astype("float32")); nq=target.predict_on_batch(S2.astype("float32")).max(1)
            tq[np.arange(bs),A]=R+gamma*nq*(1-D)
            q.train_on_batch(S.astype("float32"),tq)
        if steps%500==0: target.set_weights(q.get_weights())
    eps=max(eps_min,eps*eps_decay); rewards.append(G)
    if ep%10==0: print(f"Episode {ep:3d} | reward {G:5.0f} | avg(last 10) {np.mean(rewards[-10:]):6.1f} | epsilon
{time.time()-t0:5.0f}s",flush=True)
rewards=np.array(rewards)
# greedy evaluation
sc=[]
for k in range(20):
    s,_=env.reset(seed=100+k); done=False; G=0
    while not done:
        a=int(np.argmax(fast_q(s.reshape(1,-1).astype("float32")).numpy()[0])); s,r,term,trunc,_=env.step(a); don
trunc; G+=r
    sc.append(G)
print("\nGreedy evaluation over 20 episodes:",sc)
print("Mean evaluation reward:",np.mean(sc))
print("Success rate (reward >= 195):",np.mean(np.array(sc)>=195)*100,"%")
ma=np.convolve(rewards,np.ones(10)/10,mode="valid")
fig,ax=plt.subplots(1,2,figsize=(10,3.6))
ax[0].plot(rewards,alpha=.5,label="Episode reward"); ax[0].plot(np.arange(9,len(rewards)),ma,label="Moving avg (1
ax[0].set_title("DQN on CartPole-v1"); ax[0].set_xlabel("Episode"); ax[0].legend()
ax[1].bar(range(1,21),sc); ax[1].axhline(195,color="r",ls="--",label="Solved threshold (195)"); ax[1].set_title("
evaluation (20 episodes)"); ax[1].set_xlabel("Evaluation episode"); ax[1].legend(fontsize=8)
plt.tight_layout(); plt.savefig("figs/exp10.png",dpi=150)
```

Output

```
Episode  10 | reward    31 | avg(last 10)   20.2 | epsilon 0.860 |     1s
Episode  20 | reward    19 | avg(last 10)   15.8 | epsilon 0.739 |     1s
Episode  30 | reward    10 | avg(last 10)   14.4 | epsilon 0.635 |     2s
Episode  40 | reward     9 | avg(last 10)   11.8 | epsilon 0.546 |     2s
Episode  50 | reward     9 | avg(last 10)   11.9 | epsilon 0.470 |     2s
Episode  60 | reward    16 | avg(last 10)   12.9 | epsilon 0.404 |     2s
Episode  70 | reward    12 | avg(last 10)   14.0 | epsilon 0.347 |     3s
Episode  80 | reward    10 | avg(last 10)   11.6 | epsilon 0.298 |     3s
Episode  90 | reward    10 | avg(last 10)   11.5 | epsilon 0.257 |     3s
Episode 100 | reward     9 | avg(last 10)   10.9 | epsilon 0.221 |     3s
Episode 110 | reward    15 | avg(last 10)    9.9 | epsilon 0.190 |     3s
Episode 120 | reward    10 | avg(last 10)   10.8 | epsilon 0.163 |     4s
Episode 130 | reward    10 | avg(last 10)   10.5 | epsilon 0.140 |     4s
Episode 140 | reward    14 | avg(last 10)   11.7 | epsilon 0.121 |     4s
Episode 150 | reward    10 | avg(last 10)   11.2 | epsilon 0.104 |     4s
Episode 160 | reward    10 | avg(last 10)    9.7 | epsilon 0.089 |     5s
Episode 170 | reward    35 | avg(last 10)   21.0 | epsilon 0.077 |     5s
Episode 180 | reward    66 | avg(last 10)   34.6 | epsilon 0.066 |     6s
Episode 190 | reward   135 | avg(last 10)  117.6 | epsilon 0.057 |     9s
Episode 200 | reward   164 | avg(last 10)  150.2 | epsilon 0.049 |    12s
```

```
Episode 210 | reward   187 | avg(last 10)  188.4 | epsilon 0.042 |    16s
Episode 220 | reward   212 | avg(last 10)  181.3 | epsilon 0.036 |    20s
Episode 230 | reward   199 | avg(last 10)  199.2 | epsilon 0.031 |    25s
Episode 240 | reward   161 | avg(last 10)  194.1 | epsilon 0.027 |    29s
Episode 250 | reward   272 | avg(last 10)  184.8 | epsilon 0.023 |    33s
Episode 260 | reward   190 | avg(last 10)  189.7 | epsilon 0.020 |    37s
Episode 270 | reward   214 | avg(last 10)  199.4 | epsilon 0.020 |    42s
Episode 280 | reward   214 | avg(last 10)  193.2 | epsilon 0.020 |    46s
Episode 290 | reward   177 | avg(last 10)  177.4 | epsilon 0.020 |    50s
Episode 300 | reward   187 | avg(last 10)  159.0 | epsilon 0.020 |    53s
... (output truncated)
```

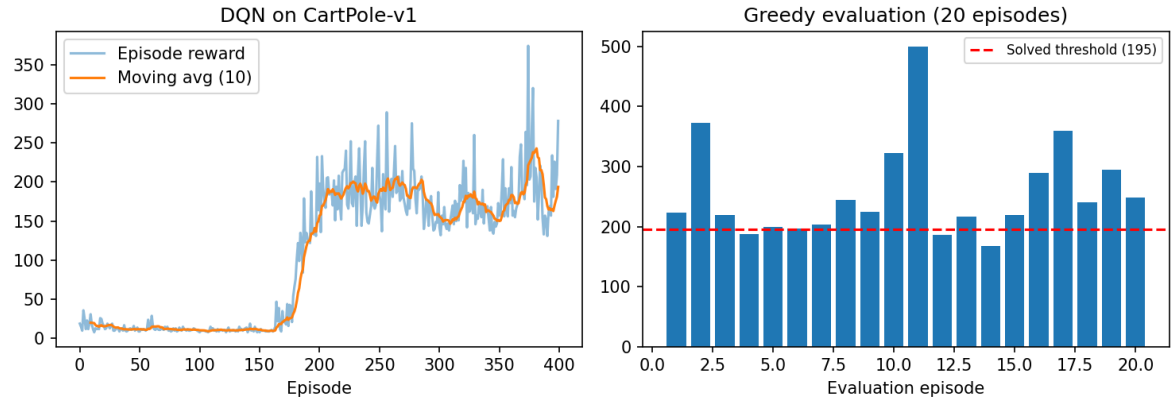


Figure 10: Output graph of Experiment 10

Result

The DQN agent learned to balance the pole after about 180 episodes. During greedy evaluation it achieved a mean reward of 255.7 and a success rate of 85% (episodes with reward >= 195), so the agent successfully solved the CartPole task in most episodes.

Code of Conduct:

*I **Aryan.K.S (192325051)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: